

JAVA

06

Spring Framework

Move beyond raw Servlets — learn how Spring's IoC, DI, and MVC turn complex Java apps into clean, maintainable, industry-ready code.

DURATION	LEVEL	MODULE	UPDATED
7 Sessions	Intermediate	Module 6	June 2026

IN THIS MODULE

01	Introduction to Spring & its Architecture
02	Inversion of Control (IoC) & Dependency Injection (DI)
03	Spring Bean Configuration
04	Annotation-Based Configuration
05	Spring JDBC Integration
06	Spring MVC Introduction
07	Mini Project — Spring MVC Module Integration

SESSION

01

60 MIN · FUNDAMENTALS

Introduction to Spring & its Architecture

◆ Learning Objectives

- Explain the limitations of raw Servlets that led to the creation of Spring Framework
- Identify the core modules of the Spring ecosystem and the purpose of each
- Describe the four-layer architecture of a Spring application
- Set up a Maven-based Spring project with the spring-context dependency
- Distinguish between BeanFactory and ApplicationContext and when each is used

01 Why Spring? The Limits of Raw Servlets

In Module 3 you built working web apps with raw Servlets and JDBC. That code worked — but every Servlet you wrote was tightly wired: it created its own database connections with 'new', instantiated service objects manually, and duplicated boilerplate across dozens of classes. As an application grows — imagine a campus placement portal handling registrations, job listings, email notifications, and payments all at once — maintaining 50 such Servlets becomes a nightmare. Spring Framework was created specifically to solve this.

CONCEPT — Spring Framework

Spring is an open-source Java application framework built around an IoC (Inversion of Control) Container. Instead of your classes creating their own dependencies using 'new', Spring creates and injects them automatically. This produces loose coupling, easy unit testing, and clean separation of concerns — the qualities that distinguish junior code from production-grade enterprise Java at companies like Infosys, Wipro, and every top-tier Ahmedabad IT firm.

SPRING ADOPTION AMONG JAVA DEVELOPERS**~60%**

Source: Snyk JVM Ecosystem Report 2023 — approximately 60% of professional Java developers use Spring or Spring Boot as their primary framework

02 The Spring Ecosystem — Eight Core Modules

Spring is not a single JAR — it is an ecosystem of modules. You include only what your project needs. A fintech company like Zerodha might use Spring Core for DI, Spring Security for auth, and Spring Data JPA for the database layer — all from the same ecosystem, all integrating without conflicts. Understanding which module handles which concern is the first skill a Spring developer builds.

Module	What It Does	When You Need It
Spring Core & Beans	IoC Container, BeanFactory, core DI mechanism	Every Spring project — this is the foundation
Spring Context	ApplicationContext, events, i18n, resource loading	Every Spring project — builds on Core
Spring AOP	Aspect-Oriented Programming for logging, transactions	Cross-cutting concerns across all layers
Spring JDBC	JdbcTemplate simplifies JDBC boilerplate by 70%	SQL-first data access without a full ORM
Spring ORM	Integrates Hibernate/JPA with Spring transactions	Used alongside Hibernate (Module 5)
Spring MVC	DispatcherServlet, Controllers, Views, ModelAndView	Building web applications with Java
Spring Security	Authentication, authorization, CSRF protection	Any app with login or role-based access
Spring Boot	Auto-configuration and embedded Tomcat server	Rapid project bootstrap — covered in Module 7

PRO TIP — Module 6 Scope

This module focuses on Spring Core, Spring JDBC, and Spring MVC — the three pillars that everything else in the Spring ecosystem builds on. Master these here and Spring Boot (Module 7) will feel like a natural upgrade, not a new technology. Every annotation, every configuration pattern, and every debugging technique you learn now applies directly in Spring Boot.

03 Spring Application Architecture — The Four-Layer Model

Every production Spring application follows a strict layered architecture: Presentation Layer (JSP, Thymeleaf) -> Service Layer (business logic) -> Repository / DAO Layer (database access) -> Database. Each layer depends only on the layer immediately below it. This architecture is standard across every Java project at large IT services firms — TCS, Wipro, and HCL all train their freshers on exactly this pattern.

Raw Servlet Approach

One Servlet reads the request, runs JDBC queries directly, builds the response HTML, and handles errors. Business logic, data access, and presentation are tangled together. A schema change in the DB forces edits in the Servlet itself.

Spring Layered Approach

Controller handles only the HTTP request. Service handles only business logic. DAO handles only database access. Each class has one responsibility. Changing the database layer has zero impact on the Controller or JSP.

04 Setting Up Spring With Maven

Adding spring-context to pom.xml is all it takes to pull in Spring Core, Spring Beans, Spring AOP, and Spring Context via Maven's transitive dependency system. You declare what you need; Maven resolves the full dependency tree automatically. This is the same workflow used in every Java project at companies like Persistent Systems and Mphasis — no manual JAR downloads, no classpath errors.

```
<!-- Add inside <dependencies> in pom.xml -->
<dependency>
  <groupId>org.springframework</groupId>
  <artifactId>spring-context</artifactId>
  <version>6.1.6</version>
</dependency>
```

Maven dependency block — paste this into your project
Open dependency tag
Spring's Maven group identifier
Pulls in Core, Beans, Context, and AOP transitively
Latest stable Spring 6.x as of June 2026
Maven downloads Spring JARs automatically on next build

TRY THIS LIVE

Create a new Maven project in IntelliJ. Add the spring-context dependency above to pom.xml and let Maven sync. Create a class Main with a main() method. Type 'ClassPathXmlApplicationContext' — IntelliJ should auto-suggest the import from spring-context. If the import resolves, your Spring JAR is on the classpath and you are ready for Session 02.

WATCH — VIDEO REFERENCE (HINDI/ENGLISH)

Channel — NPTEL (official — IITs / IISc)

Watch — Programming in Java — complete playlist by Prof. Debasis Samanta, IIT Kharagpur. Watch Lecture 1 for the Java platform and ecosystem overview before moving into Spring-specific content.

Link — <https://nptel.ac.in/courses/106105191>

Always link the official channel — beware fan-made or re-upload copies.

WHERE TO GO NEXT

You now know what Spring is and how its modules fit together. Next session goes inside the engine room — the IoC Container and Dependency Injection.

Explore deeper — Read the Spring Framework Overview at docs.spring.io/spring-framework/docs/current/reference/html/overview.html — focus on the section 'What We Mean by the Spring Framework'

Try on your own — Draw the Spring module ecosystem on paper. Place Spring Core at the centre and draw arrows to every module that depends on it. Add the business problem each module solves.

Connect the dots — Session 02 covers IoC and Dependency Injection — the single concept that separates Spring developers from Servlet developers. Everything else in Spring builds on this.

Going further — Read Martin Fowler's foundational article 'Inversion of Control Containers and the Dependency Injection Pattern' at martinfowler.com/articles/injection.html — this is the source document for all modern IoC frameworks.

AI PROMPT — ASK CHATGPT / GEMINI / CLAUDE

Act as a Java mentor. In simple Hinglish, explain what Spring Framework is and why it was needed, using a Paytm-style digital wallet app as the example. Show concretely how the Servlet code becomes messy without Spring and how Spring fixes it. Keep it under 150 words and avoid jargon.

SESSION

02

60 MIN · CORE CONCEPTS

Inversion of Control (IoC) & Dependency Injection (DI)

◆ Learning Objectives

- Define Inversion of Control and explain why it reduces coupling between classes
- Distinguish between Constructor Injection and Setter Injection and when to use each
- Write an applicationContext.xml file with bean declarations and dependency wiring
- Load the Spring IoC Container using ClassPathXmlApplicationContext
- Retrieve a fully wired bean from the container and invoke its methods

01 The Problem — Tight Coupling

Tight coupling happens when class A creates class B itself using 'new B()'. This means A cannot function without B's exact implementation — swap B for a different class and you must edit A too. Imagine an IRCTC booking system where every module — TrainService, PaymentService, NotificationService — directly instantiates each other. One change in PaymentService cascades through the entire codebase. This is the problem IoC was designed to eliminate.

CONCEPT — Inversion of Control (IoC)

IoC means handing control of object creation to an external container rather than letting each class create its own dependencies. In Spring, the IoC Container reads your configuration, creates all bean objects, and injects them wherever needed. You say 'I need a PaymentService' — Spring delivers it. The key inversion: the container calls your code, not your code calling the container.

Without IoC — Tight Coupling

```
class BookingService {
    PaymentService ps = new PaymentService();
}
```

BookingService is permanently tied to PaymentService. Writing a unit test for BookingService requires a real PaymentService — and a real database.

With IoC — Loose Coupling

```
class BookingService {
    PaymentService ps; // Spring injects this
}
```

BookingService works with any PaymentService implementation. In tests, inject a MockPaymentService. In production, inject the RazorpayPaymentService. Zero code change.

02 Dependency Injection — Two Approaches

Spring supports two primary DI mechanisms: Constructor Injection and Setter Injection. Both achieve the same goal — your class receives the objects it needs — but they differ in when the dependency is provided and how mandatory it is. Choosing the right one for each scenario is a key skill in professional Spring development.

DI Type	How It Works	Best Used When
Constructor Injection	Dependency is passed as a constructor argument when the bean is created	Dependency is mandatory; the class cannot function without it
Setter Injection	Spring calls a setXxx() method after creating the bean	Dependency is optional or has a sensible default value
Field Injection (@Autowired)	Spring injects directly into the field (annotation-based only)	Quick prototyping only — avoid in production; breaks testability

03 Writing Your First applicationContext.xml

Before annotations, Spring used an XML file called applicationContext.xml to declare beans and their dependencies. Learning XML config is important — it makes the 'magic' of annotation-based config fully transparent. When you debug a Spring context error, knowing what the XML equivalent looks like tells you exactly what Spring is trying to do under the hood.

```
<beans xmlns="http://www.springframework.org/schema/beans"
  Root element — declares the Spring XML namespace
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  XML Schema reference for validation
  xsi:schemaLocation="http://www.springframework.org/schema/beans
  Schema location for IDE auto-complete support
  http://www.springframework.org/schema/beans/spring-beans.xsd">
  Closing the schema location attribute
  <bean id="studentRepo" class="com.tops.StudentRepository"/>
  Declares StudentRepository as a Spring-managed bean
  <bean id="studentService" class="com.tops.StudentService">
  Declares StudentService — Spring will create this object
    <constructor-arg ref="studentRepo"/>
    Constructor injection — Spring passes studentRepo
    into StudentService
  </bean>
  Close the studentService bean declaration
</beans>
  Close the root element
```

The ApplicationContext is Spring's main IoC Container. You bootstrap it by pointing it to your XML file. From that point, Spring owns the full lifecycle of every declared bean — creation, wiring, and destruction. You simply ask the container for a bean by ID and get back a fully wired object, with all its dependencies already injected.

```
// Main.java — bootstrap the Spring container
ApplicationContext ctx =
new ClassPathXmlApplicationContext("applicationContext.xml");
  Spring reads XML from src/main/resources, creates all beans
  Entry point to start the IoC Container
  Declare the ApplicationContext variable
```

```
StudentService svc = (StudentService) ctx.getBean("studentService");
```

Retrieve the bean — StudentRepository is already injected

```
svc.enrollStudent("Ravi Patel", "Java Full Stack");
```

Call business method — no 'new' keyword anywhere in this file

05 Setter Injection in XML

Setter injection uses the property tag. Spring first creates the bean using its no-arg constructor, then calls the setter method. This is ideal for optional dependencies — like a notification service that enhances the core feature but is not required for it to function. If no notifier is configured, the service still works; Spring simply skips the injection.

```
<bean id="emailNotifier" class="com.tops.EmailNotificationService"/>
```

Optional dependency — sends confirmation emails

```
<bean id="studentService" class="com.tops.StudentService">
```

Core service bean

```
<property name="notifier" ref="emailNotifier"/>
```

Setter injection — Spring calls setNotifier(emailNotifier)

```
</bean>
```

If property tag is removed, StudentService still works without notifier

CASE STUDY — Razorpay's Payment Provider Architecture

Context — Razorpay, India's leading payment gateway (founded in Bengaluru in 2014), processes payments for over 8 million businesses including Zomato, OYO, and Cred.

Challenge — Razorpay needed to support multiple payment providers — UPI, credit cards, net banking, wallets — without rewriting the core transaction processing logic every time a new provider was onboarded.

What they did — They defined a PaymentProcessor interface and wired the correct provider implementation at runtime via their IoC container. Each provider (UPI, card, wallet) was a separate Spring bean registered with a unique ID.

Result — Onboarding a new payment provider required only writing a new class that implements PaymentProcessor and registering it as a bean — zero changes to the core transaction service.

Takeaway — IoC and DI are not theoretical patterns. They are the architecture that allows Indian fintech companies to scale their payment options without rewriting core business logic.

Source: *Razorpay Engineering Blog*, razorpay.com/blog — articles on payment orchestration architecture (2022)

TRY THIS LIVE

Create StudentRepository with a findAll() method returning a hardcoded list. Create StudentService with a constructor that accepts StudentRepository and a method getAllStudents() that delegates to the repo. Write

applicationContext.xml wiring them via constructor injection. In Main.java, load the context and call getAllStudents(). Verify there is no 'new StudentRepository()' anywhere in StudentService.

WHERE TO GO NEXT

You can now wire beans manually in XML. Next you will learn how Spring manages bean instances — whether it creates one per request or reuses a single shared instance.

Explore deeper — Spring docs — 'The IoC Container' at docs.spring.io/spring-framework/docs/current/reference/html/core.html#beans

Try on your own — Swap the constructor injection in your demo for setter injection. Then delete the setter from StudentService and observe what exception Spring throws — and read the message carefully.

Connect the dots — Session 03 covers bean scopes and lifecycle hooks — controlling exactly when Spring creates and destroys your beans.

Going further — Read about the BeanFactory vs ApplicationContext distinction in the Spring docs. BeanFactory is lazy (creates beans on demand); ApplicationContext is eager (creates all singletons on startup). Understanding this explains startup time differences in large Spring apps.

SESSION

03

60 MIN · CORE CONCEPTS

Spring Bean Configuration

◆ Learning Objectives

- Define a Spring Bean and explain its role in the IoC Container lifecycle
- Configure beans in applicationContext.xml with properties, constructor args, and values
- Differentiate between singleton and prototype bean scopes and choose correctly for each use case
- Apply autowiring modes (byName, byType, constructor) to reduce XML verbosity
- Implement bean lifecycle callbacks using init-method and destroy-method

01 What Is a Spring Bean?

A Spring Bean is any Java object whose lifecycle — creation, configuration, and destruction — is managed by the Spring IoC Container. You do not use 'new' to create it. You declare it in XML or with an annotation, and Spring handles everything else. Think of the container as a hotel concierge at a five-star hotel in Ahmedabad: you request a room service; the concierge creates, prepares, and delivers it — you never go to the kitchen yourself.

CONCEPT — Bean Lifecycle

Every Spring bean goes through a fixed lifecycle: 1) Container reads bean definition (XML or annotation). 2) Bean is instantiated using the constructor. 3) Dependencies are injected. 4) init-method is called (if defined). 5) Bean is in use. 6) Container shuts down — destroy-method is called (if defined). Understanding this lifecycle lets you initialize database connections or thread pools at step 4 and close them cleanly at step 6.

02 Bean Scopes — How Many Instances Spring Creates

Bean scope controls how many instances of a bean Spring creates and how they are shared. The most common scopes are singleton and prototype. A wrong scope choice is one of the most common Spring bugs in production — a singleton bean holding request-specific data can cause data leaks across user sessions. This is a real issue that has hit Indian e-commerce portals during peak sale events.

Scope	Instances Created	Use Case
singleton (default)	One per ApplicationContext — shared by all callers	Stateless services, repositories, utility classes
prototype	A new instance on every getBean() call	Stateful objects, user-specific data holders
request	One per HTTP request (web apps only)	Request-scoped data in Spring MVC controllers

Scope	Instances Created	Use Case
session	One per HTTP session (web apps only)	Shopping cart, user login state per session
application	One per ServletContext (web apps only)	App-wide configuration shared across all sessions

```
<!-- Singleton – Spring creates ONE instance shared everywhere -->
```

Default scope; no need to declare explicitly, shown here for clarity

```
<bean id="studentService" class="com.tops.StudentService" scope="singleton"/>
```

All callers get the same StudentService instance

```
<!-- Prototype – Spring creates a NEW instance on every getBean() call -->
```

Use for stateful beans that should not be shared

```
<bean id="orderForm" class="com.tops.OrderForm" scope="prototype"/>
```

Each call to ctx.getBean('orderForm') returns a fresh object

PRO TIP — Singleton is Not Thread-Safe by Default

A singleton bean is shared across all threads. If your service bean holds any instance variables that change during a request — like a 'currentUserId' field — multiple concurrent users will overwrite each other's data. Keep your singleton beans stateless: pass all request-specific data as method parameters, never as instance fields.

03 Autowiring — Letting Spring Wire Dependencies Automatically

Autowiring reduces XML verbosity by letting Spring automatically match beans to dependencies without you writing an explicit ref attribute for each one. Spring matches by the bean ID (byName), by the bean type (byType), or via the constructor. For small projects this speeds up config significantly; for large projects, explicit wiring is often preferred for clarity.

```
<!-- byName – Spring looks for a bean whose ID matches the property name -->
```

Property name in Java class must match the bean ID exactly

```
<bean id="studentService" class="com.tops.StudentService" autowire="byName"/>
```

If StudentService has a 'studentRepo' property, Spring injects the bean with id='studentRepo'

```
<!-- byType – Spring looks for a bean whose type matches the property type -->
```

Fails with an error if there are two beans of the same type

```
<bean id="studentService" class="com.tops.StudentService" autowire="byType"/>
```

Spring injects whichever bean implements StudentRepository — only works if exactly one exists

```
<!-- constructor — matches constructor arguments by type -->
```

Used when you prefer constructor injection but want Spring to match automatically

```
<bean id="studentService" class="com.tops.StudentService" autowire="constructor"/>
```

Spring matches constructor arg types to available beans

CONCEPT — When Autowiring Fails

Autowiring byType throws a NoUniqueBeanDefinitionException if two beans of the same type exist in the context — for example, two classes that implement StudentRepository. The fix is to either use byName instead, or mark one bean as primary using the primary='true' attribute. Understanding this exception is important because it is one of the most common errors beginners hit when working on real multi-module Spring projects.

04 Bean Lifecycle Hooks — init-method and destroy-method

Spring lets you hook into the bean lifecycle with custom initialization and destruction methods. The init-method runs right after Spring has injected all dependencies — ideal for opening database connections or loading configuration caches. The destroy-method runs when the application context is closed — ideal for releasing resources. This prevents resource leaks in long-running server applications.

```
<!-- Declare lifecycle methods in XML -->
```

Spring calls these methods automatically at the right moment

```
<bean id="dbConnectionPool" class="com.tops.DatabaseConnectionPool"
```

Bean that manages a pool of DB connections

```
init-method="initPool"
```

Spring calls initPool() after all dependencies are injected

```
destroy-method="closePool"/>
```

Spring calls closePool() when the ApplicationContext is closed — use ctx.close() in main()

TRY THIS LIVE

Add init-method='start' and destroy-method='stop' to your StudentService bean in applicationContext.xml. Implement start() and stop() in StudentService with a println each. In main(), call ctx.close() after using the service. Run and observe the output — you will see Spring call start() on boot and stop() on close. Change scope to 'prototype' and verify that stop() is NOT called automatically for prototypes (Spring does not manage prototype destruction).

WATCH — VIDEO REFERENCE (HINDI/ENGLISH)

Channel — NPTEL (official — IITs / IISc)

Watch — Programming in Java — Lecture series by Prof. Debasis Samanta, IIT Kharagpur. Relevant for Java object lifecycle concepts that underpin Spring bean management.

Link — <https://nptel.ac.in/courses/106105191>

WHERE TO GO NEXT

You can now declare beans, set their scopes, and hook into their lifecycle in XML. Next you replace all of this XML with clean, readable Java annotations.

Explore deeper — Spring docs — 'Customizing the Nature of a Bean' at docs.spring.io/spring-framework/docs/current/reference/html/core.html#beans-factory-nature

Try on your own — Create three different `StudentService` beans in `applicationContext.xml` — one singleton, one prototype — and call `getBean()` twice for each. Print the object's `hashCode`. Confirm that the singleton returns the same `hashCode` each time and the prototype returns a different one.

Connect the dots — Session 04 replaces XML bean configuration entirely with `@Component`, `@Service`, `@Repository`, and `@Autowired` annotations — the style used in every modern Spring project.

Going further — Explore the `InitializingBean` and `DisposableBean` interfaces — Spring's alternative to `init-method/destroy-method` that wires lifecycle directly into the class. Compare the trade-offs between the two approaches for production code.

SESSION

04

60 MIN · CORE CONCEPTS

Annotation-Based Configuration

◆ Learning Objectives

- Explain why annotation-based configuration replaced XML in modern Spring projects
- Apply `@Component`, `@Service`, `@Repository`, and `@Controller` stereotype annotations correctly
- Use `@Autowired` for constructor, setter, and field injection
- Define beans programmatically using `@Configuration` and `@Bean`
- Enable component scanning with `<context:component-scan>` or `@ComponentScan`

01 From XML to Annotations — Why the Shift Happened

Large XML configuration files became a maintenance problem as Spring projects grew. A mid-size enterprise project at an IT services company in Pune might have 50 to 100 bean declarations spread across multiple XML files. Annotations moved the configuration right next to the code it configures. Adding a new class means annotating it — no XML file to update, no bean ID to remember, no sync issues. This is why annotation-based config is now the industry standard and what every Spring Boot project uses by default.

XML Configuration (Sessions 02–03)

Declare every bean in `applicationContext.xml`.
Configuration is in one place but disconnected from the class. Adding a new service requires editing XML.
Typos in class names fail at runtime, not compile time.

Annotation Configuration (This Session)

Annotate the class directly — `@Service` on `StudentService`, `@Repository` on `StudentRepository`.
Configuration lives with the code. Refactoring the class name updates the annotation automatically. IDE support catches errors immediately.

CONCEPT — Component Scanning

Annotations alone are not enough — Spring must know which packages to scan for annotated classes. You enable this with `<context:component-scan base-package='com.tops'/>` in XML or `@ComponentScan('com.tops')` in a Java config class. Spring scans all classes in that package and its sub-packages, finds any class annotated with `@Component` (or its specializations), and registers it as a bean automatically.

02 Stereotype Annotations — @Component and Its Specializations

Spring provides four stereotype annotations that register a class as a bean. They all do the same thing technically — each is a specialization of `@Component` — but they communicate intent. Using `@Service` on a business logic class tells every developer who reads the code exactly where that class lives in the application architecture. Indian IT teams follow this convention strictly in code reviews.

Annotation	Layer	Purpose
@Component	Any	Generic bean registration — use when none of the others fit
@Service	Service Layer	Business logic classes — payment processing, enrollment logic
@Repository	DAO / Repository Layer	Database access classes — adds exception translation for DB errors
@Controller	Presentation Layer	Spring MVC web controllers — handles HTTP requests (Session 06)

```
// StudentRepository.java – DAO layer
```

```
@Repository
```

```
public class StudentRepository {
    public List<String> findAll() {
```

```
    return List.of("Ravi Patel", "Priya Shah", "Amit Desai");
```

Dummy data for now; replace with JDBC in Session 05

```
    }
}
```

Spring registers this as a bean named

'studentRepository' (camelCase of class name)

Marks this as a DAO bean; also activates Spring's DB exception translation

Plain Java class — no XML needed

Business method — Spring manages the object, you write the logic

End method

End class

```
// StudentService.java – Service layer
```

```
@Service
```

```
public class StudentService {
    private final StudentRepository repo;
    @Autowired
```

```
public StudentService(StudentRepository repo) { this.repo = repo; }
```

Constructor injection — preferred for mandatory dependencies

```
public List<String> getAllStudents() { return repo.findAll(); }
```

Delegates to repository

```
}
```

Spring registers this as a bean named 'studentService'

Marks this as a Service bean — business logic lives here

Plain Java class

Dependency — Spring will inject this

Tell Spring to inject the StudentRepository bean here

End class

PRO TIP — Prefer Constructor Injection With @Autowired

As of Spring 4.3+, if your class has only one constructor, @Autowired is optional — Spring injects automatically. But always write it explicitly for clarity. Constructor injection is preferred over field injection because it makes dependencies visible in the API, allows the class to be instantiated without Spring in unit tests, and enables final fields for immutability.

03 @Configuration and @Bean — Programmatic Bean Definitions

For beans you cannot annotate directly — third-party library classes, beans that require complex construction logic, or beans with environment-specific settings — Spring provides `@Configuration` and `@Bean`. A `@Configuration` class is a factory class: each `@Bean` method creates and returns one bean, and Spring manages its lifecycle just like any XML-declared bean.

<code>@Configuration</code>	Marks this class as a source of Spring bean definitions
<code>public class AppConfig {</code>	Java config class — replaces applicationContext.xml entirely
 <code>@Bean</code>	Declares a Spring bean — method name becomes the bean ID
 <code>public DataSource dataSource() {</code>	Bean ID = 'dataSource' — Spring calls this method to create the bean
 <code>DriverManagerDataSource ds = new DriverManagerDataSource();</code>	
<code>Third-party class — we cannot add @Component to it</code>	
<code>ds.setDriverClassName("com.mysql.cj.jdbc.Driver");</code>	
<code>Configure the datasource properties</code>	
<code>ds.setUrl("jdbc:mysql://localhost:3306/tops_campus");</code>	
<code>Connection URL for TOPS Campus Connect DB</code>	
<code>ds.setUsername("root");</code>	DB username
<code>ds.setPassword("yourpassword");</code>	DB password — use environment variables in production
 <code>return ds;</code>	Spring stores this object and injects it wherever DataSource is needed
<code>}</code>	End @Bean method
<code>}</code>	End @Configuration class

CONCEPT — @Bean vs @Component

`@Component` (and its specializations) is used on classes you own and can modify. `@Bean` is used inside a `@Configuration` class when you need to create a bean from a class you cannot annotate — like a third-party `DriverManagerDataSource` or a `JdbcTemplate` from Spring's own library. In practice you will use both in every real project: `@Service/@Repository` for your classes, `@Bean` for framework and library objects.

When using `@Configuration` classes, you replace `ClassPathXmlApplicationContext` with `AnnotationConfigApplicationContext`. This loads your Java config class instead of an XML file. In Spring Boot (Module 7), this bootstrap is done automatically — but understanding the manual version makes Spring Boot's auto-configuration much easier to debug.

```
// Main.java — load Spring context from @Configuration class
```

No XML file needed when using annotation config

```
ApplicationContext ctx = new AnnotationConfigApplicationContext(AppConfig.class);
```

Spring reads AppConfig.class, runs @Bean methods, scans @Component classes

```
StudentService svc = ctx.getBean(StudentService.class);
```

Retrieve by type instead of by string ID — safer, refactoring-friendly

```
svc.getAllStudents().forEach(System.out::println);
```

Spring has already injected StudentRepository into StudentService

```
((AnnotationConfigApplicationContext) ctx).close();
```

Close the context — triggers @PreDestroy or destroy-method callbacks

CASE STUDY — Naukri.com Modular Spring Architecture

Context — Naukri.com (Info Edge India, Noida) is India's largest job portal with over 70 million registered users and processes millions of job applications and recruiter searches daily.

Challenge — As the platform grew from a monolith to a multi-module application, maintaining XML configuration files across dozens of modules became error-prone. A missing or mistyped bean ID in XML caused runtime startup failures that were hard to trace.

What they did — Naukri migrated their Java backend modules to annotation-based Spring configuration, combining @Service, @Repository, and @Configuration classes with component scanning per module package. Each module became a self-contained Spring configuration unit.

Result — Onboarding new modules became faster, configuration errors were caught at compile time by IDEs, and the codebase became significantly easier for new engineers to navigate.

Takeaway — Annotation-based configuration is not just a style preference — it is a practical productivity tool that reduces configuration errors and speeds up onboarding on large Java teams.

Source: Info Edge India Engineering Blog and LinkedIn tech talks, 2021–2023

TRY THIS LIVE

Delete applicationContext.xml from your project. Create AppConfig.java with @Configuration and @ComponentScan("com.tops"). Add @Service to StudentService and @Repository to StudentRepository (remove any @Autowired XML refs). In Main.java, switch to AnnotationConfigApplicationContext(AppConfig.class). Run — if everything works without a single line of XML, you have successfully converted your project to annotation config.

WHERE TO GO NEXT

Your project is now XML-free. The next step is to wire Spring to a real MySQL database using JdbcTemplate.

Explore deeper — Spring docs — '@Configuration Classes' at docs.spring.io/spring-framework/docs/current/reference/html/core.html#beans-java

Try on your own — Create two classes that both implement an interface — say, SmsSender and

EmailSender both implement NotificationSender. Annotate both with @Component. Try autowiring NotificationSender and observe the NoUniqueBeanDefinitionException. Fix it with @Primary on one of them.

Connect the dots — Session 05 uses @Configuration to define a DataSource bean and wires it into a JdbcTemplate for real CRUD operations on MySQL.

Going further — Explore @Qualifier — it lets you pick a specific bean when multiple beans of the same type exist, without making one 'primary'. This is the production-grade solution to the ambiguity problem.

SESSION

05

60 MIN · DATA ACCESS

Spring JDBC Integration

◆ Learning Objectives

- Explain how JdbcTemplate eliminates JDBC boilerplate code
- Configure a DataSource bean using @Configuration and inject it into JdbcTemplate
- Perform all four CRUD operations on a MySQL table using JdbcTemplate
- Implement RowMapper to convert ResultSet rows into Java objects
- Build a complete DAO layer using JdbcTemplate that follows Spring conventions

01 From Raw JDBC to JdbcTemplate

In Module 2 you wrote raw JDBC code: create connection, create statement, execute query, iterate ResultSet, close statement, close connection — and wrap everything in try-catch-finally. That is roughly 20 lines for a single SELECT query. JdbcTemplate reduces that to 2 lines. It handles connection acquisition, statement creation, exception translation, and resource cleanup internally. You write only the SQL and the mapping logic.

Raw JDBC — 18 Lines for One Query

```
Connection con = DriverManager.getConnection(url,
user, pass);
PreparedStatement ps = con.prepareStatement(sql);
ResultSet rs = ps.executeQuery();
while (rs.next()) { ... }
rs.close(); ps.close(); con.close();
// All of this inside a try-catch-finally block
```

JdbcTemplate — 2 Lines for One Query

```
return jdbcTemplate.query(
"SELECT * FROM students",
new StudentRowMapper());
// Connection, statement, ResultSet, cleanup — all
handled by Spring
```

CONCEPT — JdbcTemplate

JdbcTemplate is Spring's central class for JDBC data access. It wraps a DataSource, acquires connections from it, executes your SQL, translates checked SQLExceptions into Spring's unchecked DataAccessException hierarchy, and releases resources automatically. You never call connection.close() manually. Every DAO in a Spring JDBC project holds one JdbcTemplate instance, injected by Spring at startup.

02 Configuring DataSource and JdbcTemplate

The DataSource is the connection factory. JdbcTemplate uses it to get connections on demand. Both are configured as Spring beans in your @Configuration class. In production, the DataSource is typically a connection pool like HikariCP, but for learning purposes DriverManagerDataSource is sufficient. This same configuration pattern is used in every Spring JDBC project at IT firms in Ahmedabad and across India.

@Configuration

Java config class — defines all infrastructure beans

public class DatabaseConfig {

Separate config class for database concerns

@Bean

DataSource bean — Spring manages this as a singleton

public DataSource dataSource() {

Returns a DataSource — JdbcTemplate will use this to get connections

DriverManagerDataSource ds = new DriverManagerDataSource();

Simple DataSource — creates a new connection per request (for learning)

ds.setDriverClassName("com.mysql.cj.jdbc.Driver");

MySQL 8 driver class

ds.setUrl("jdbc:mysql://localhost:3306/campus_connect");

Connection URL for Campus Connect database

ds.setUsername("root");

DB username

ds.setPassword("yourpassword");

DB password — never hardcode in production

return ds;

Spring stores this and injects it wherever DataSource is needed

}**@Bean**

JdbcTemplate bean — wraps the DataSource

public JdbcTemplate jdbcTemplate(DataSource dataSource) {

Spring auto-injects the DataSource bean defined above

return new JdbcTemplate(dataSource);

All DAO classes will get this singleton JdbcTemplate injected

}**}**

End DatabaseConfig

03 CRUD Operations With JdbcTemplate

JdbcTemplate has three main method groups. `update()` handles INSERT, UPDATE, and DELETE — operations that modify data. `query()` and `queryForObject()` handle SELECT — returning lists or single objects. All methods accept parameterized SQL with `?` placeholders, preventing SQL injection. This is the same safe pattern used in the JDBC module — Spring just automates the plumbing.

JdbcTemplate Method	SQL Operation	Returns
<code>update(sql, args...)</code>	INSERT, UPDATE, DELETE	int — number of rows affected
<code>query(sql, rowMapper, args...)</code>	SELECT — multiple rows	List<T> mapped by your RowMapper
<code>queryForObject(sql, rowMapper, args...)</code>	SELECT — exactly one row	T — single mapped object (throws if 0 or 2+ rows)

JdbcTemplate Method	SQL Operation	Returns
queryForObject(sql, type, args...)	SELECT — single value (COUNT, SUM)	Integer, Long, String, etc.
batchUpdate(sql, batchArgs)	Bulk INSERT/UPDATE	int[] — rows affected per batch entry

```
// EventDao.java — all four CRUD operations
// Full DAO for the Campus Connect Portal's Event feature

@Repository
public class EventDao {
    @Autowired
    // Marks this as a Spring-managed DAO bean
    private JdbcTemplate jdbcTemplate;

    // CREATE
    public int addEvent(String title, String date, int hostUserId) {
        // Returns rows inserted (1 on success)
        String sql = "INSERT INTO events (title, event_date, host_user_id) VALUES (?, ?, ?)";
        // Parameterised SQL — Spring passes args safely
        return jdbcTemplate.update(sql, title, date, hostUserId);
        // Spring acquires connection, executes, closes — you write no boilerplate
    }

    // READ ALL
    public List<Event> getAllEvents() {
        // Returns all events as a List<Event>
        return jdbcTemplate.query("SELECT * FROM events", new EventRowMapper());
        // EventRowMapper converts each ResultSet row into an Event object
    }

    // UPDATE
    public int updateEventTitle(int id, String newTitle) {
        // Returns rows updated
        return jdbcTemplate.update("UPDATE events SET title=? WHERE id=?", newTitle, id);
    }

    // DELETE
    public int deleteEvent(int id) {
        // Returns rows deleted
        return jdbcTemplate.update("DELETE FROM events WHERE id=?", id);
    }
}
// End EventDao
```

04 RowMapper — Mapping ResultSet to Java Objects

JdbcTemplate's query() method needs a RowMapper to know how to convert a database row into a Java object. You implement the RowMapper interface and write the mapping once. Spring calls your mapRow() method for every row in the ResultSet. This eliminates the manual rs.getString(), rs.getInt() loop you wrote in raw JDBC — and more importantly, it isolates the mapping in one testable class.

```
// EventRowMapper.java — maps one DB row to one Event object
// Implements Spring's RowMapper<Event> interface
public class EventRowMapper implements RowMapper<Event> {
    @Override
    public Event mapRow(ResultSet rs, int rowNum) throws SQLException {
        // Spring calls this for each row — rowNum is the row index
        Event event = new Event();           // Create a fresh Event object for this row
        event.setId(rs.getInt("id"));         // Map the 'id' column to the Event id field
        event.setTitle(rs.getString("title"));
        // Map the 'title' column
        event.setEventDate(rs.getString("event_date"));
        // Map the 'event_date' column
        event.setHostUserId(rs.getInt("host_user_id"));
        // Map the foreign key column
        return event;                         // Spring adds this to the result List<Event>
    }
}
```

End EventRowMapper

CASE STUDY — Zerodha Kite's Data Access Layer

Context — Zerodha (Bengaluru) is India's largest stockbroker by active clients, handling over 15 million orders on peak trading days through its Kite platform.

Challenge — Zerodha's order management system needed to execute thousands of DB writes per second during market hours. Raw JDBC code across dozens of service classes was causing connection leaks and inconsistent error handling, leading to occasional data integrity issues.

What they did — The team standardized on a Spring JDBC DAO layer using JdbcTemplate with a HikariCP connection pool. All database access was centralized in @Repository DAO classes, with consistent exception handling via Spring's DataAccessException hierarchy.

Result — Connection leaks were eliminated. Error handling became consistent and testable. The DAO layer could be tested independently of the web layer using Spring's test context support.

Takeaway — JdbcTemplate is not just a convenience — it enforces consistent patterns that prevent resource leaks and unhandled exceptions, which matter enormously in high-throughput financial systems.

Source: Zerodha Z-Connect Blog and Zerodha engineering team discussions on GitHub (2021–2023)

TRY THIS LIVE

Create an 'events' table in your local MySQL: `CREATE TABLE events (id INT AUTO_INCREMENT PRIMARY KEY, title VARCHAR(100), event_date VARCHAR(20), host_user_id INT)`. Write `EventDao` with `addEvent()` and `getAllEvents()` using `JdbcTemplate`. Write `EventRowMapper`. Wire everything in `DatabaseConfig`. Call `addEvent('Java Hackathon', '2026-07-15', 1)` from `main()`, then call `getAllEvents()` and print each event. Verify the data in MySQL Workbench.

WHERE TO GO NEXT

Your backend can now read from and write to a real database via Spring JDBC. Next you build the web layer — handling HTTP requests and rendering dynamic JSP pages.

Explore deeper — Spring docs — 'Data Access with JDBC' at docs.spring.io/spring-framework/docs/current/reference/html/data-access.html#jdbc

Try on your own — Add a `countEvents()` method to `EventDao` using `jdbcTemplate.queryForObject("SELECT COUNT(*) FROM events", Integer.class)`. Test it before and after inserting a row.

Connect the dots — Session 06 introduces Spring MVC — adding a Controller layer that sits in front of the Service and DAO layers you have built in Sessions 02–05.

Going further — Explore Spring's `NamedParameterJdbcTemplate` — it replaces positional ? parameters with named :paramName parameters, making complex multi-parameter SQL statements much more readable and maintainable.

AI PROMPT — ASK CHATGPT / GEMINI / CLAUDE

Act as a Spring JDBC tutor. In simple Hinglish, explain what JdbcTemplate does and why it is better than raw JDBC, using an Ahmedabad e-commerce store's order management system as the example. Show the difference in code length for a simple SELECT query. Keep it under 150 words.

SESSION

06

60 MIN · WEB LAYER

Spring MVC Introduction

◆ Learning Objectives

- Explain the role of DispatcherServlet as the front controller in Spring MVC
- Describe the request lifecycle through HandlerMapping, Controller, ViewResolver, and View
- Configure Spring MVC using web.xml and dispatcher-servlet.xml
- Write a @Controller class with @RequestMapping for GET and POST requests
- Return data from a Controller to a JSP View using ModelAndView

01 From Raw Servlets to Spring MVC

In Module 3, every URL in your web app mapped to a different Servlet. Adding ten new pages meant writing ten Servlet classes, each with its own `doGet()` and `doPost()`, each manually forwarding to JSPs. Spring MVC replaces this with a single entry-point Servlet — the `DispatcherServlet` — that routes all requests to the correct Controller method based on URL patterns. This is the same architectural shift that powers every major Indian web portal, from IRCTC to BookMyShow.

CONCEPT — DispatcherServlet (Front Controller Pattern)

The `DispatcherServlet` is a single Servlet that receives all HTTP requests for your application and delegates them to the appropriate handler. It uses `HandlerMapping` to find the right `@Controller` method for each URL, invokes the method, receives a `ModelAndView`, uses a `ViewResolver` to find the JSP, merges the model data with the JSP, and sends the final HTML response. One Servlet replaces dozens — that is the Front Controller pattern.

02 Spring MVC Request Lifecycle

Understanding the request lifecycle is essential for debugging Spring MVC applications. When something goes wrong — a 404, a model attribute missing in JSP, or a redirect loop — knowing which step in the lifecycle failed tells you exactly where to look. This lifecycle is the same whether you are building a simple form or a full REST API.

Step	Component	What Happens
1	Browser / Client	HTTP request arrives — e.g. GET /events
2	DispatcherServlet	Receives the request — acts as the Front Controller

Step	Component	What Happens
3	HandlerMapping	Maps the URL /events to <code>EventController.listEvents()</code> method
4	@Controller Method	Executes business logic, fetches data from Service/DAO layer
5	ModelAndView	Controller returns view name and model data (the events list)
6	ViewResolver	Translates view name 'events' to <code>/WEB-INF/views/events.jsp</code>
7	JSP View	Renders HTML by merging model data with the JSP template
8	Browser / Client	Receives final HTML — the events page is displayed

```
<!-- web.xml – register DispatcherServlet (WEB-INF/web.xml) -->
```

This is the only Servlet you need to declare in web.xml for Spring MVC

```
<servlet>
```

```
  <servlet-name>dispatcher</servlet-name>
```

Name used to link servlet and servlet-mapping below

```
  <servlet-class>org.springframework.web.servlet.DispatcherServlet</servlet-class>
```

Spring's Front Controller Servlet

```
  <load-on-startup>1</load-on-startup>
```

Initialize DispatcherServlet at app startup, not on first request

```
</servlet>
```

```
<servlet-mapping>
```

```
  <servlet-name>dispatcher</servlet-name>
```

```
  <url-pattern>/</url-pattern>
```

All requests go through DispatcherServlet — '/' means every URL

```
</servlet-mapping>
```

03 Writing Your First @Controller

A Spring MVC Controller is a plain Java class annotated with `@Controller`. Each method in the class handles a specific URL. The `@RequestMapping` annotation maps a URL pattern (and optionally HTTP method) to a Java method. The method does its work, packages results into a `ModelAndView` object, and returns. Spring MVC handles everything else.

@Controller

Marks this class as a Spring MVC Controller — beans are auto-registered

```
public class EventController {
```

@Autowired

Spring injects the EventService bean

```
private EventService eventService;
```

Service layer — Controller never calls DAO directly

```
@RequestMapping(value = "/events", method = RequestMethod.GET)
```

Handles GET /events

```
public ModelAndView listEvents() {
```

Method name is arbitrary — URL mapping comes from @RequestMapping

```
List<Event> events = eventService.getAllEvents();
```

Get data from Service layer

```
ModelAndView mav = new ModelAndView("events");
```

View name 'events' — ViewResolver maps this to /WEB-INF/views/events.jsp

```
mav.addObject("eventList", events);
```

Add data to model — accessible in JSP as \${eventList}

```
return mav;
```

DispatcherServlet renders the JSP with the model data

```
}
```

```
}
```

End EventController

PRO TIP — Why Controllers Should Not Call DAOs

A common beginner mistake is to @Autowire a DAO directly into a Controller and skip the Service layer. Avoid this. The Controller's job is only to translate HTTP into Java and Java back into HTTP. Business logic belongs in the Service layer. If you put business logic in the Controller, it cannot be reused — a batch job or a scheduled task cannot call an HTTP Controller.

04 Handling Form Submissions — POST Requests

Spring MVC maps form fields to method parameters automatically using @RequestParam. You declare the parameter name matching the form field name and Spring extracts the value from the HTTP POST body. This is the same mechanism that powers every registration form, login page, and search bar in Spring MVC applications across India's enterprise IT landscape.

```
// Handle the 'Add Event' form submission in EventController
```

Second method in the same @Controller class

```
@RequestMapping(value = "/addEvent", method = RequestMethod.GET)
```

Shows the empty Add Event form

```
public String showAddEventForm() {
```

String return type — shorthand for returning just a view name

```
return "addEvent";
```

Renders /WEB-INF/views/addEvent.jsp

```
}
```

```
@RequestMapping(value = "/addEvent", method = RequestMethod.POST)
```

Handles form submission from addEvent.jsp

```
public String processAddEvent(@RequestParam String title,
```

@RequestParam maps the 'title' form field to this parameter

```
@RequestParam String eventDate,
```

@RequestParam maps the 'eventDate' form field

```
@RequestParam int hostUserId) {
```

Spring auto-converts the String form value to int

```
eventService.addEvent(title, eventDate, hostUserId);
```

Delegate to Service — Controller never writes SQL

```
return "redirect:/events";
```

Post/Redirect/Get pattern — prevents double-submit on browser refresh

```
}
```

End processAddEvent

CASE STUDY — BookMyShow's Spring MVC Web Layer

Context — BookMyShow (Bigtree Entertainment, Mumbai) is India's largest online ticketing platform, selling over 200 million tickets per year across movies, events, and sports.

Challenge — BookMyShow's original Servlet-based architecture had over 200 individual Servlets, each managing its own request routing and response logic. Adding a new booking flow required writing new Servlets, updating web.xml, and redeploying. Developers spent more time on plumbing than features.

What they did — They migrated to a Spring MVC architecture with a centralized DispatcherServlet, organizing feature-specific Controllers (@Controller) by domain — MovieController, EventController, SportsController — each backed by Service and DAO layers.

Result — New booking flows could be added as new @RequestMapping methods in existing Controllers without touching web.xml. The codebase became modular, testable, and much faster to iterate on during peak sale periods.

Takeaway — Spring MVC's Front Controller pattern is not just cleaner code — it is the architecture that allows high-traffic Indian product companies to ship new features rapidly without rewriting infrastructure.

Source: BookMyShow Engineering Blog at inside.bookmyshow.com and public conference talks by BookMyShow engineers (2019–2022)

TRY THIS LIVE

In dispatcher-servlet.xml, configure an InternalResourceViewResolver with prefix="/WEB-INF/views/" and suffix=".jsp". Create EventController with listEvents() returning ModelAndView('events') containing a dummy list. Create /WEB-INF/views/events.jsp that iterates \${eventList} using JSTL forEach. Deploy to Tomcat and hit <http://localhost:8080/yourapp/events>. Verify you see the event list rendered in HTML without writing a single Servlet.

WHERE TO GO NEXT

Your application now has a complete web layer. The final session brings everything together into a full Spring MVC mini project for the Campus Connect Portal.

Explore deeper — Spring docs — 'Web MVC framework' at docs.spring.io/spring-framework/docs/current/reference/html/web.html

Try on your own — Add a search form to your events page — a GET form with a 'keyword' parameter. Write a `searchEvents(@RequestParam String keyword)` method in `EventController` that filters the events list. Render results in the same `events.jsp`.

Connect the dots — Session 07 is the module mini project — you combine all six sessions into a full Spring MVC application with Controller, Service, DAO, and JSP layers working together.

Going further — Explore `@ModelAttribute` — instead of mapping individual `@RequestParam` fields, you can map an entire HTML form to a Java bean in one annotation. This is the standard approach for forms with many fields in production Spring MVC projects.

WATCH — VIDEO REFERENCE (HINDI/ENGLISH)

Channel — NPTEL (official — IITs / IISc)

Watch — Internet Technologies — Web application architecture lectures by IIT Kharagpur. Relevant for understanding the client-server and MVC concepts underlying Spring MVC.

Link — <https://nptel.ac.in/courses/106105084>

SESSION

07

60 MIN · PROJECT

Mini Project — Spring MVC Module Integration

◆ Learning Objectives

- Convert the Campus Connect Portal's Event Management feature from raw Servlets to Spring MVC
- Apply all six sessions of Module 6 in a single end-to-end application
- Implement a four-layer Spring architecture: Controller, Service, DAO, and JSP View
- Wire all layers using @Autowired annotation-based dependency injection
- Deploy the completed Spring MVC module to Tomcat and demonstrate full CRUD via browser

01 What You Will Build — Campus Connect Portal (Spring MVC Edition)

In Modules 3 and 4, you built the Campus Connect Portal using raw Servlets and Hibernate. In this mini project, you take the Event Management feature and rebuild it from scratch using the full Spring MVC stack from this module. The result is a clean, four-layer web application — the same architecture pattern used at every IT company that hires Java Full Stack developers in Ahmedabad, Bengaluru, and Pune.

PROJECT SCOPE — Campus Connect Event Manager

Build a web-based Event Management module where students can: view all upcoming events (list page), add a new event via a form (create page), update event details (edit page), and delete an event. The stack: Spring MVC (@Controller) for the web layer -> EventService (@Service) for business logic -> EventDao (@Repository + JdbcTemplate) for database access -> events.jsp, addEvent.jsp, editEvent.jsp for the view layer.

Feature	URL	HTTP Method	Controller Method
List all events	/events	GET	listEvents() -> events.jsp
Show Add Event form	/events/add	GET	showAddForm() -> addEvent.jsp
Save new event	/events/add	POST	saveEvent() -> redirect:/events
Show Edit Event form	/events/edit?id={id}	GET	showEditForm() -> editEvent.jsp
Update event	/events/edit	POST	updateEvent() -> redirect:/events
Delete event	/events/delete?id={id}	GET	deleteEvent() -> redirect:/events

02 Project Structure and Maven Setup

A professional Spring MVC project follows a Maven standard directory layout. Java source files go under `src/main/java`, web resources under `src/main/webapp`. Keeping this structure consistent allows every developer on the team — and every IDE — to understand the project without reading documentation. This is the structure you will see on your first day at any Java IT firm.

<code>campus-connect-spring/</code>	Root project folder
<code>+- pom.xml</code>	Maven build file — spring-webmvc, spring-jdbc, mysql-connector dependencies
<code>+- src/main/</code>	
<code>+- java/com/tops/</code>	All Java source code under the com.tops package
<code>+- config/</code>	
<code>+- AppConfig.java</code>	@Configuration + @ComponentScan — replaces applicationContext.xml
<code>+- DatabaseConfig.java</code>	@Configuration — defines DataSource and JdbcTemplate beans
<code>+- controller/</code>	
<code>+- EventController.java</code>	@Controller — handles all /events/* URLs
<code>+- service/</code>	
<code>+- EventService.java</code>	@Service — business logic, input validation
<code>+- dao/</code>	
<code>+- EventDao.java</code>	Interface — defines CRUD contract
<code>+- EventDaoImpl.java</code>	@Repository — JdbcTemplate implementation
<code>+- model/</code>	
<code>+- Event.java</code>	POJO — id, title, eventDate, hostUserId, venue
<code>+- webapp/WEB-INF/</code>	
<code>+- web.xml</code>	Registers DispatcherServlet to handle all requests
<code>+- dispatcher-servlet.xml</code>	Spring MVC config — ViewResolver, component-scan
<code>+- views/</code>	
<code>+- events.jsp</code>	Lists all events in a Bootstrap table with Edit/Delete links
<code>+- addEvent.jsp</code>	Form with title, date, venue, hostUserId fields
<code>+- editEvent.jsp</code>	Pre-populated form for updating an existing event

03 Step-by-Step Implementation

Build the project bottom-up: Model first, then DAO, then Service, then Controller, then JSP. This order ensures every layer is complete and tested before the layer above it is built. It also mirrors how professional development teams structure their sprint tasks in a Java Full Stack project.

<code>-- Step 1: Create the events table in MySQL</code>	Run this SQL in MySQL Workbench before writing any Java
<code>CREATE DATABASE IF NOT EXISTS campus_connect;</code>	Create the database

```
USE campus_connect;
CREATE TABLE events (
  id          INT AUTO_INCREMENT PRIMARY KEY,
  title       VARCHAR(150) NOT NULL,
  event_date  DATE NOT NULL,
  venue       VARCHAR(200),
  host_user_id INT NOT NULL
);
```

Auto-incrementing primary key
Event title — required field
Event date
Location or online link
Foreign key to users table (simplified as int here)
Table ready — now build the Java layers

Step 2 — Build the Event POJO and EventDao interface. Step 3 — Implement EventDaoImpl with JdbcTemplate for all four CRUD operations and an EventRowMapper. Step 4 — Create EventService with @Service, inject EventDao, and add a validateEvent() method that checks for empty title and past dates. Step 5 — Write EventController with all six URL mappings from the table in Section 01.

```
// EventController.java – key methods (abbreviated for reference)
Full implementation built during session
@Controller
@RequestMapping("/events")

public class EventController {
  @Autowired EventService eventService;

  @GetMapping("") // GET /events
  Shorthand for @RequestMapping(method=GET)
  public ModelAndView list() {
    return new ModelAndView("events", "eventList", eventService.getAllEvents());
  }
  Shorthand constructor: view name, model key, model value

  @PostMapping("/add") // POST /events/add
  Handles form submission
  public String save(@ModelAttribute Event event) {
    @ModelAttribute binds all form fields to the Event POJO automatically
    eventService.addEvent(event);
    return "redirect:/events";
  }
}
```

Class-level mapping — all methods handle URLs starting with /events
Spring injects EventService — no 'new' needed
Service validates and calls DAO
PRG pattern — redirect to list after save
End EventController — Edit and Delete methods follow same pattern

Step 6 — Build the three JSP views. Use JSTL c:forEach to iterate the event list in events.jsp. Use c:url for action attributes to keep URLs context-path safe. For editEvent.jsp, pre-populate form fields using the 'event' model attribute. Step 7 — Deploy to Tomcat, test all six URLs manually, then verify the database state in MySQL Workbench after each operation.

```
<!-- events.jsp — main list view (key snippet) -->
Full JSP built during session
<%@ taglib prefix="c" uri="http://java.sun.com/jsp/jstl/core" %>
JSTL core taglib — required for c:forEach, c:url
<table class="table table-striped">                                Bootstrap 5 table for clean styling
<thead><tr><th>ID</th><th>Title</th><th>Date</th><th>Actions</th></tr></thead>
Table header
<tbody>
  <c:forEach var="e" items="${eventList}">                        Iterate the List<Event> from ModelAndView
    <tr>
      <td>${e.id}</td>                                              Event ID from the model
      <td>${e.title}</td>                                           Event title
      <td>${e.eventDate}</td>                                       Event date
      <td>
        <a href="/events/edit?id=${e.id}">Edit</a> |
        Link to Edit form, passing event ID
        <a href="/events/delete?id=${e.id}">Delete</a>
        Delete link — triggers deleteEvent() in Controller
      </td>
    </tr>
  </c:forEach>
</tbody>
</table>
Clean, working table rendered by Spring MVC
```

04 Demonstration Checklist

DEMO CHECKLIST — What to Show Your Trainer

1. Open <http://localhost:8080/campus-connect/events> — the event list loads from MySQL. 2. Click 'Add Event', fill the form, submit — new row appears in the list and in MySQL Workbench. 3. Click 'Edit' on any event, change the title, submit — list updates. 4. Click 'Delete' on any event — row disappears from list and database. 5. In IntelliJ, show that EventController has zero 'new' keywords — all dependencies are Spring-injected. 6. Show that EventService has no JDBC code — it only calls EventDao. This demonstrates a clean, production-style four-layer Spring MVC application.

SPRING MVC IN INDIA JOB LISTINGS

Top 3

Source: Naukri.com Java Full Stack job analysis (2024) — Spring MVC is in the top 3 required skills in 85% of Java Full Stack job descriptions on India's top job portals

WHERE TO GO NEXT

You have built a complete four-layer Spring MVC application. Module 7 takes everything you know and runs it through Spring Boot — auto-configuration replaces your XML and Java config, and an embedded Tomcat replaces manual deployment.

Explore deeper — Read the Spring MVC documentation chapter 'Annotated Controllers' at docs.spring.io/spring-framework/docs/current/reference/html/web.html#mvc-controller — it covers `@ModelAttribute`, `@SessionAttributes`, and `@ControllerAdvice` for error handling.

Try on your own — Add server-side validation to the Add Event form — if the title is empty or the date is in the past, return the `addEvent.jsp` with an error message instead of saving. Use an if-check in `EventService` and pass the error as a model attribute.

Connect the dots — Module 7 — Spring Boot — starts with the same Campus Connect Portal and converts it to a Spring Boot project. You will see how auto-configuration eliminates your `DatabaseConfig`, `web.xml`, and `dispatcher-servlet.xml` in under 10 minutes.

Going further — Explore `@ControllerAdvice` and `@ExceptionHandler` — they let you define one central class to handle exceptions thrown from any Controller. Without this, every Controller method needs its own try-catch. This is the Spring MVC equivalent of a global error handler.

REVIEW



INTERVIEW PREP · MODULE 6

Interview Questions

◆ Spring

Rehearse each answer out loud and aim for a 60-second response. Use the hint only as a nudge if you are stuck. Compare your attempt with the model answer afterward to find gaps.

1

What is the Spring Framework, and what problem does it aim to solve for Java developers?

Think about how Spring changes the way Java applications are structured and managed.

MODEL ANSWER

Spring is an open-source framework for building enterprise Java applications. It solves the problem of tight coupling and boilerplate code that plague plain Java EE applications. Before Spring, developers had to manually manage object creation, lifecycle, and dependencies. Spring introduces an IoC container that creates and wires objects for you, promotes loose coupling through dependency injection, and provides integrations for MVC, JDBC, security, and more — all configurable without writing repetitive infrastructure code.

2

Explain the main differences between traditional Servlets/JSP development and using the Spring Framework.

Focus on how configuration, object creation, and modularity are handled.

MODEL ANSWER

In plain Servlets/JSP, you manually instantiate DAOs and services inside each servlet using 'new', configure everything in web.xml, and there is no standard way to manage dependencies. Spring replaces this with an IoC container that creates and injects objects automatically, annotation-based or XML configuration, a DispatcherServlet that centralises request routing, and clean separation of controller, service, and repository layers. Code becomes modular, testable, and much less repetitive.

3

Name any two core modules of the Spring Framework and briefly describe their purpose.

MODEL ANSWER

Spring Core provides the IoC container and dependency injection mechanism — the foundation all other modules build on. Spring MVC provides a model-view-controller web framework on top of Servlets, letting you build web applications and REST APIs with annotations like @Controller and @RequestMapping. Other important modules include Spring JDBC (simplifies JDBC operations), Spring Security (authentication/authorisation), and Spring Data (repository abstractions). Each module can be used independently or combined.

4

What is Inversion of Control (IoC) in the context of Spring?*Consider who is responsible for creating and managing objects.***MODEL ANSWER**

In traditional code, your class creates its own dependencies using 'new'. IoC inverts this — you declare what your class needs, and the Spring container creates and provides those dependencies for you. The container reads your configuration (XML or annotations), builds all the beans, resolves dependencies, and manages their lifecycle. Your classes focus on business logic rather than wiring. IoC makes it easy to swap implementations (for example, replacing a real DAO with a mock in tests) without changing the class that uses it.

5

Why does Dependency Injection (DI) improve modularity in Java applications?*Think about how dependencies are provided to classes and how this affects testing or swapping implementations.***MODEL ANSWER**

DI means a class receives its dependencies from the outside rather than creating them internally. This decouples the class from the specific implementation it uses — a `StudentService` does not care whether its `StudentDAO` queries MySQL or H2; it just uses the DAO interface. This makes it straightforward to swap implementations for testing (inject a mock DAO) or for feature changes (swap a payment provider). Each class becomes independently testable and reusable, which is the hallmark of modular design.

6

Describe the difference between setter injection and constructor injection in Spring.*Focus on how dependencies are passed to the bean.***MODEL ANSWER**

Constructor injection provides dependencies through the class constructor — they are required and available as soon as the object is created, making the object immutable and easy to test. Setter injection provides dependencies through setter methods after construction — dependencies are optional and can be changed after creation. Spring recommends constructor injection for mandatory dependencies and setter injection for optional ones. Constructor injection also prevents circular dependency issues from being hidden at startup.

7

In a Spring application, how would you inject a dependency into a `StudentService` bean using XML configuration?**MODEL ANSWER**

In `applicationContext.xml`: `<bean id="studentDao" class="com.tops.dao.StudentDAOImpl"/> <bean id="studentService" class="com.tops.service.StudentService"> <property name="studentDao" ref="studentDao"/> </bean>`. The 'property' element uses the setter method (`setStudentDao`) to inject the reference. For constructor injection, use `<constructor-arg ref="studentDao"/>`. The `StudentService` class must have the corresponding setter or constructor for Spring to call.

8

How do you define a bean in applicationContext.xml in Spring?*Mention the main XML tag and required attributes.***MODEL ANSWER**

Use the '<bean>' element with 'id' (unique name used to retrieve the bean) and 'class' (fully qualified class name): '<bean id="myBean" class="com.tops.MyClass"/>'. The Spring container instantiates this class and makes it available under the id 'myBean'. You can inject dependencies with '<property>' (setter) or '<constructor-arg>' (constructor). The XML file is loaded by an ApplicationContext: 'new ClassPathXmlApplicationContext("applicationContext.xml")'.

9

What is the difference between singleton and prototype bean scopes in Spring?*Consider how many instances Spring creates for each scope.***MODEL ANSWER**

Singleton scope (the default) means the container creates exactly one instance of the bean and returns the same instance every time it is requested. Prototype scope means the container creates a new instance every time the bean is requested. Use singleton for stateless beans like services and DAOs (safe to share). Use prototype for stateful beans like command objects or beans that hold per-user data. Singleton beans are cached; prototype beans are created fresh on every `getBean()` call or injection.

10

Explain what autowiring is in Spring and how it helps with dependency management.**MODEL ANSWER**

Autowiring lets Spring automatically resolve and inject dependencies without requiring explicit '<property>' or '<constructor-arg>' declarations. Spring matches beans by type (byType), name (byName), or constructor. In annotation-based config, `@Autowired` on a field, constructor, or setter tells Spring to inject the matching bean automatically. This reduces XML configuration significantly. For XML, you can set 'autowire="byType"' on a bean definition. Autowiring works best when there is exactly one candidate bean of the required type.

11

If you have two beans of the same type in your Spring configuration, how does Spring decide which one to autowire?*Think about what happens if there is ambiguity.***MODEL ANSWER**

Spring throws a `NoUniqueBeanDefinitionException` because it cannot determine which bean to inject. You resolve this with `@Qualifier("beanName")` alongside `@Autowired` to specify exactly which bean to inject. Alternatively, mark one bean as the default with `@Primary` so Spring prefers it when multiple candidates exist. In XML, use the 'primary="true"' attribute. This situation arises when you have multiple `DataSource` or multiple DAO implementations and need to be explicit about which one goes where.

12

How would you configure multiple beans in Spring and access them using the IoC container?

MODEL ANSWER

Define all beans in applicationContext.xml. Load the context: `ApplicationContext ctx = new ClassPathXmlApplicationContext("applicationContext.xml")`. Retrieve a bean by its id: `StudentService service = ctx.getBean("studentService", StudentService.class)`. The container has already resolved all dependencies between the beans at startup. In a web application, the context is loaded automatically by `ContextLoaderListener` defined in web.xml, and beans are injected into servlets via Spring MVC.

13

What is the purpose of the `@Component` annotation in Spring?

Consider what happens to a class when it is annotated this way.

MODEL ANSWER

`@Component` marks a class as a Spring-managed bean so the container automatically detects and registers it during component scanning. Instead of declaring the bean in XML, you annotate the class and enable `<context:component-scan base-package="com.tops"/>`. Specialised stereotypes provide more semantic meaning: `@Service` for business logic, `@Repository` for DAO classes (also enables exception translation), and `@Controller` for MVC controllers. All four result in the class being registered as a bean.

14

How does `@Autowired` help in annotation-based configuration?

Think about how dependencies are injected automatically.

MODEL ANSWER

`@Autowired` instructs Spring to inject the matching bean into the annotated field, constructor, or setter method. On a field: `private @Autowired StudentDAO studentDao;` — Spring finds the bean of type `StudentDAO` and injects it. On a constructor: Spring injects via constructor parameters. If only one constructor exists, `@Autowired` is implied since Spring 4.3. Combined with `@Component-scanned` beans, `@Autowired` eliminates virtually all XML wiring declarations, keeping configuration close to the code.

15

What is the difference between using `@Bean` in a `@Configuration` class and using `@Component` on a class?

Focus on how beans are registered in the Spring context.

MODEL ANSWER

`@Component` (and its stereotypes) is placed on the class itself — Spring scans for it and registers it automatically. `@Bean` is placed on a method inside a `@Configuration` class — you write the method body that constructs and returns the bean, giving you full control over instantiation logic. Use `@Bean` when you cannot annotate the source class (e.g., third-party libraries), or when the construction requires complex logic. Use `@Component` for your own application classes where direct annotation is cleaner.

16

How would you convert an XML-based configuration to annotation-based configuration in a Spring project?

MODEL ANSWER

Replace each '<bean>' declaration by adding @Component (or @Service/@Repository) to the corresponding class. Replace '<property ref="...">' injections with @Autowired on the field or constructor. Enable component scanning in the remaining XML with '<context:component-scan base-package="com.tops"/>', or switch entirely to a @Configuration class with @ComponentScan. Remove the old <bean> entries. Test that the context loads and that all dependencies are injected correctly before removing the XML file.

17

Show how you would annotate a service class with @Component and inject a dependency using @Autowired.

MODEL ANSWER

'@Component public class StudentService { @Autowired private StudentDAO studentDao; public List<Student> getAllStudents() { return studentDao.findAll(); } }'. The @Component registers StudentService as a bean. @Autowired injects the StudentDAO bean (which must also be annotated @Repository or @Component). No XML is needed if component scanning covers the package. In a test, you can inject a mock StudentDAO with @MockBean (Spring Boot) or by using @Autowired with a stub implementation.

18

What is JdbcTemplate in Spring, and why is it commonly used for database operations?

Consider how it simplifies JDBC code.

MODEL ANSWER

JdbcTemplate is a Spring class that wraps JDBC and eliminates the repetitive boilerplate: obtaining a connection, creating statements, iterating ResultSets, and closing resources. You simply provide the SQL and a RowMapper that converts each row to an object. JdbcTemplate handles connection management, exception translation (converts SQLExceptions to Spring's DataAccessException hierarchy), and resource cleanup automatically. Compared to raw JDBC, a query that takes 20+ lines becomes 3-4 lines with JdbcTemplate.

19

Describe how you would configure a datasource in a Spring application for JDBC access.

MODEL ANSWER

In applicationContext.xml: '<bean id="dataSource" class="org.springframework.jdbc.datasource.DriverManagerDataSource"> <property name="driverClassName" value="com.mysql.cj.jdbc.Driver"/> <property name="url" value="jdbc:mysql://localhost:3306/tops_db"/> <property name="username" value="root"/> <property name="password" value="root"/> </bean>'. Inject this dataSource bean into your DAO or JdbcTemplate bean. In annotation-based config, declare it in a @Configuration class with a @Bean method returning a DataSource. For production, use a connection pool like HikariCP instead of DriverManagerDataSource.

20

Write a simple method using `JdbcTemplate` to insert a new `Student` record into the database.

MODEL ANSWER

'public void addStudent(Student s) { String sql = "INSERT INTO students (name, email) VALUES (?, ?)"; jdbcTemplate.update(sql, s.getName(), s.getEmail()); }'. `jdbcTemplate.update()` is used for INSERT, UPDATE, and DELETE. Parameters are passed as varargs — Spring substitutes them as `PreparedStatement` parameters, preventing SQL injection. The method returns the number of rows affected. Inject `JdbcTemplate` via `@Autowired`; configure it as a Spring bean with the `DataSource` injected.

21

How would you organize the DAO layer in a Spring application using Spring JDBC?

MODEL ANSWER

Define a DAO interface (e.g., `StudentDAO`) with methods like `addStudent()`, `findById()`, `findAll()`, `updateStudent()`, `deleteStudent()`. Create an implementation class (`StudentDAOImpl`) annotated `@Repository` that holds an `@Autowired JdbcTemplate`. Each method contains a SQL string and a call to `jdbcTemplate.update()` or `jdbcTemplate.query()` with a `RowMapper` lambda. The `@Service` layer depends only on the `StudentDAO` interface, not the implementation. This structure supports easy mocking in tests and clean separation between business logic and data access.

22

What steps are involved in performing a CRUD operation (like update or delete) on a `Student` table using Spring JDBC?

MODEL ANSWER

For update: `'jdbcTemplate.update("UPDATE students SET name=? WHERE id=?", student.getName(), student.getId())'`. For delete: `'jdbcTemplate.update("DELETE FROM students WHERE id=?", id)'`. Both return the row count. For read: `'jdbcTemplate.queryForObject("SELECT * FROM students WHERE id=?", new StudentRowMapper(), id)'` maps a single row to a `Student` object. For a list: `'jdbcTemplate.query("SELECT * FROM students", new StudentRowMapper())'`. The `RowMapper` implements `mapRow(ResultSet rs, int rowNum)` to build the object.

23

Explain the role of `DispatcherServlet` in Spring MVC.

Think about what happens when a web request comes in.

MODEL ANSWER

`DispatcherServlet` is the front controller in Spring MVC — all incoming HTTP requests pass through it first. It consults the `HandlerMapping` to find which `@Controller` method matches the URL, calls that method, receives the `ModelAndView` or response body, resolves the view name using a `ViewResolver` (which maps 'student/list' to '/WEB-INF/views/student/list.jsp'), and forwards the rendered output to the client. It centralises request dispatching, eliminating the need for each `Servlet` to handle routing logic independently.

24

What is the purpose of a Controller in Spring MVC?*Consider how requests are processed and responses generated.***MODEL ANSWER**

A Controller is a class annotated `@Controller` that handles specific HTTP requests mapped to it by `@RequestMapping`. It reads request parameters or path variables, calls the service layer to process the business logic, puts results into a Model (or `ModelMap`), and returns a view name for the `DispatcherServlet` to resolve and render. Controllers are lightweight — they should not contain business logic or SQL, just orchestrate the flow between the request, service, and view. For REST APIs, `@RestController` combines `@Controller` and `@ResponseBody` to return JSON directly.

25

How does Spring MVC handle GET and POST requests differently in a controller method?**MODEL ANSWER**

Use `@GetMapping("/students")` (or `@RequestMapping(method=RequestMethod.GET)`) to handle GET requests — typically used for retrieving and displaying data. Use `@PostMapping("/students")` for POST requests — typically used for form submissions that create or update data. GET handler methods usually populate a Model and return a view name. POST handler methods validate input, call the service to persist data, and often redirect to a GET URL after success (`'redirect:/students'`) to prevent form resubmission on browser refresh (Post-Redirect-Get pattern).

26

What is ModelMap in Spring MVC, and how is it used in controllers?*Think about sharing data between controller and view.***MODEL ANSWER**

`ModelMap` is a map of attributes that the controller populates with data to pass to the view. You add data with `'model.addAttribute("students", studentList)'`. The `DispatcherServlet` makes all model attributes available as request attributes, so the JSP can access them with EL expressions like `'${students}'`. `ModelMap` can be declared as a method parameter in the controller method — Spring injects an empty map automatically. Alternatively, return a `ModelAndView` object that packages both the model data and the view name together.

27

Describe the flow of a simple 'Add Event' form submission in a Spring MVC application.**MODEL ANSWER**

The user opens the form (GET `/events/add`) — the controller method returns the view name `'addEvent'`. The user fills the form and submits (POST `/events/add`). The POST controller method receives an Event object via `@ModelAttribute` (Spring binds form fields to the object's fields), calls `eventService.save(event)`, and redirects to `'redirect:/events'` to show the updated list. If validation fails, it re-renders the form view with error messages added to the Model. The service calls the DAO, which uses `JdbcTemplate` or `JPA` to persist the record.

28

Show how you would map a URL to a controller method using `@RequestMapping` in Spring MVC.

MODEL ANSWER

```
@Controller @RequestMapping("/students") public class StudentController {
    @GetMapping public String listStudents(ModelMap model) {
        model.addAttribute("students", studentService.findAll());
        return "student/list";
    }
    @GetMapping("/{id}") public String viewStudent(@PathVariable int id, ModelMap model) {
        model.addAttribute("student", studentService.findById(id));
        return "student/detail";
    }
}
```

Class-level `@RequestMapping` sets a base path; method-level `@GetMapping` refines it. `@PathVariable` binds the URL segment to a parameter.

29

When integrating a feature like 'Event Management' into a Spring MVC module, which annotations would you typically use in your controller class?

MODEL ANSWER

`@Controller` (marks it as a web controller), `@RequestMapping('/events')` (base URL), `@GetMapping` and `@PostMapping` (method-specific mappings), `@ModelAttribute` (binds form data to an Event object), `@PathVariable` (extracts IDs from URLs like `/events/5`), `@Autowired` (injects the `EventService`), and `@SessionAttributes` (if you need to carry form data across requests). For validation, add `@Valid` on the model attribute parameter and `BindingResult` to capture errors.

30

How would you render a JSP view from a Spring MVC controller after processing form data?

MODEL ANSWER

The controller method returns a `String` — the logical view name like `'event/list'`. The configured `InternalResourceViewResolver` prepends a prefix (`'/WEB-INF/views/'`) and appends a suffix (`'.jsp'`), resolving it to `'/WEB-INF/views/event/list.jsp'`. `DispatcherServlet` forwards the request (with model attributes) to that JSP. The JSP uses JSTL and EL to render the data. Configure the resolver in the Spring XML config or via `@Bean`: `'resolver.setPrefix("/WEB-INF/views/"); resolver.setSuffix(".jsp")'`.

31

Imagine you are building the 'Event Management' module for Campus Connect using Spring MVC. Briefly outline the main components and how they interact.

MODEL ANSWER

`EventController` (`@Controller`) handles HTTP requests. For `GET /events`, it calls `EventService.getAllEvents()`, adds the list to the Model, and returns the `'event/list'` view. For `POST /events/add`, it takes an `@ModelAttribute Event`, calls `EventService.save(event)`, and redirects. `EventService` (`@Service`) contains business logic — validation, coordinating multiple DAO calls. `EventDAO` (`@Repository`) uses `JdbcTemplate` for database queries. Views (`/WEB-INF/views/event/*.jsp`) render the data using JSTL. The `DispatcherServlet` routes all requests. Spring's IoC wires all components together via `@Autowired`.

32

Suppose your Spring MVC form is not displaying validation errors as expected. What steps would you take to troubleshoot the issue?

Think about controller logic, model attributes, and the JSP view.

MODEL ANSWER

First, confirm `@Valid` is on the `@ModelAttribute` parameter and `BindingResult` immediately follows it — if the order is wrong, Spring redirects instead of re-rendering. Check that the controller returns the form view name (not a redirect) when `'bindingResult.hasErrors()'` is true. In the JSP, verify that `'<form:errors path="fieldName"/>'` uses the Spring form tag library and the correct field names matching the bean's properties. Enable Spring debug logging to see which validation constraints are failing. Check that the validation dependency (hibernate-validator) is in the classpath.

33

How would you use annotation-based configuration to wire up both your DAO and service layers in a Spring MVC mini-project?

MODEL ANSWER

Annotate the DAO class with `@Repository` and inject `JdbcTemplate` via `@Autowired`. Annotate the service class with `@Service` and inject the DAO interface via `@Autowired`. Annotate the controller with `@Controller` and inject the service via `@Autowired`. Enable component scanning in your Spring config (XML or `@Configuration` class) to cover all packages. Spring creates all three beans on startup, detects their `@Autowired` dependencies, and injects them in the correct order. No XML bean definitions are needed for the DAO, service, or controller themselves.

34

If you have to switch from XML-based bean configuration to annotation-based configuration in an existing Spring MVC module, what are the key changes you would make?

MODEL ANSWER

Add `@Component/@Service/@Repository` to each class that previously had a `<bean>` declaration. Add `@Autowired` wherever `<property ref="...">` injections existed. In the XML file, replace all `<bean>` entries with a single `<context:component-scan base-package="com.tops"/>`. Keep only infrastructure beans in XML (`DataSource`, `ViewResolver`, `JdbcTemplate`) since these often need explicit configuration. Test that the Spring context loads and all beans are wired. You can also create a `@Configuration` class and replace the XML entirely using `@Bean` methods and `@ComponentScan`.

35

Explain how you would handle form validation in a Spring MVC controller.

MODEL ANSWER

Add validation annotations to the model class fields (`@NotNull`, `@Size`, `@Email` from `javax.validation`). In the controller POST method, add `@Valid` before the `@ModelAttribute` parameter and a `BindingResult` parameter immediately after: `'public String save(@Valid @ModelAttribute Event event, BindingResult result, ModelMap model)'`. If `'result.hasErrors()'` is true, re-render the form view with the errors. In the JSP, use `'<form:errors path="name"/>'` to display field-level messages. Add the Hibernate Validator dependency to `pom.xml` for the validation annotations to work.

36

How could you use an AI tool like ChatGPT or GitHub Copilot to help you write or debug a Spring bean configuration, and how would you ensure the generated code is correct?

Think about verifying code output and checking against official documentation.

MODEL ANSWER

ChatGPT can suggest @Bean configurations, explain why a NoSuchBeanDefinitionException is occurring, or generate a JdbcTemplate DAO from a table schema. Copilot can autocomplete @Autowired fields and suggest RowMapper implementations. Before using the output, verify the class names and package paths match your project, check that the Spring version in the suggestion matches your pom.xml (APIs changed between Spring 4, 5, and 6), and run the application to confirm the context loads without errors. Cross-check against Spring's official documentation for any configuration you are unfamiliar with.

37

In what ways could AI tools help speed up development when building a Spring MVC project, and what are some things you should always double-check before using AI-generated code in production?

MODEL ANSWER

AI tools can generate repetitive CRUD controllers, DAO methods, RowMapper implementations, and XML or annotation configurations in seconds. They can explain stack traces, suggest fixes for common Spring errors, and scaffold a full module from a table schema. Before using in production: verify SQL uses parameters (not string concatenation) to prevent injection, confirm exception handling is appropriate, ensure no credentials are hardcoded, check that the bean scope (singleton vs. prototype) matches your use case, and run all unit and integration tests. AI accelerates scaffolding but does not replace understanding.

38

Integrative: How does Dependency Injection make it easier to test components in a Spring MVC application?

Consider how dependencies can be swapped or mocked.

MODEL ANSWER

Because the StudentService receives its StudentDAO through injection (not by creating it with 'new'), in a JUnit test you can construct a StudentService and inject a mock DAO: 'StudentService service = new StudentService(mockDao)'. The test does not need a database; the mock DAO returns predefined data. This is impossible when the service instantiates its own DAO internally. With Spring's @MockBean or Mockito, each layer can be tested in isolation. DI is the key enabler of unit testing in Spring applications.

39

Integrative: When performing CRUD operations on a Student table, how would you use annotation-based configuration to wire up your DAO and service beans?

MODEL ANSWER

@Repository StudentDAOImpl holds an @Autowired JdbcTemplate. @Service StudentService holds an @Autowired StudentDAO (interface). @Controller StudentController holds an @Autowired StudentService. Component scanning registers all three. Spring injects JdbcTemplate (which needs the DataSource — defined via @Bean or XML) into the DAO, injects the DAO into the service, and injects the service into the controller. Each class only knows about the interface it depends on, not the concrete implementation, enabling clean testing and future flexibility.

40

Integrative: Explain how the DispatcherServlet works together with beans defined in your application context to process a web request.

MODEL ANSWER

When a request arrives, DispatcherServlet consults the HandlerMapping bean (which scans @RequestMapping annotations) to find the correct controller method. It calls that method, which uses @Autowired service and DAO beans (managed by the ApplicationContext) to process the data. The method returns a view name; DispatcherServlet passes it to the ViewResolver bean, which maps it to a JSP file. The JSP renders the Model attributes as HTML, and DispatcherServlet returns the response. All components — DispatcherServlet, controllers, services, DAOs, ViewResolver — are beans managed by the same Spring ApplicationContext.